ENP RACING TEAM

VERSION 1.1 · 11 Oct 2026

aero_auto 1.1 Technical Guide

Multi-element wing design, simulation and reporting toolchain built on JavaFoil. Dark edition - code, reasons and wiring for every addition.

This report documents the code as it exists today: the topology, the design decisions and their measured rationale, and - for every feature added - the exact source, why it exists, how it works and what it is for. Source snippets are extracted verbatim from the modules at build time, so the document can never drift from the code.

JavaFoil-driven multi-element wing stack
CLI and GUI on the same engine
Linux, Windows and macOS

Contents

1	System architecture
2	Design decisions and rationale
3	aero_auto.py - the engine
4	aero_targets.py - the target report
5	flowfield.py - the schematic potential-flow model
6	aero_report.py - reporting pipeline
7	aero_ui.py - the threaded GUI
8	render_field.py + launcher.py - entry points
9	Validation and performance
10	Packaging and build
11	Changelog - everything added and why

1 System architecture

The toolchain is seven flat modules that import each other by file name (there is no package install step). The GUI and the CLI launcher share the same engine, so a wing you sanity-check in the GUI can be run as a campaign and then reported by the same code path.

Module	Responsibility
aero_auto.py	Regulatory defaults, geometry/outlines, JavaFoil runner (run, diagnose, parse), quality scoring, multi-phase campaign planner, force/geometry output columns, all-mm rule checks.
aero_targets.py	Target report - every valid row compared with the CL*A design target and the CD*A budget roof.
aero_report.py	Report builder - ranked tables, dark-free Excel workbook, legend, polar/geometry/field graphs, legacy-CSV repair, optional denser JavaFoil polar re-run (--resim).
flowfield.py	Schematic potential-flow model (source panels + quarter-chord vortices + ground images) for the field images.
aero_ui.py	Tkinter shell - configure/sanity/simulate/campaigns/Graphs tab, internal airfoil store, queue-based threading, icon support.
render_field.py	CLI to render a single field PNG from a wing configuration.
launcher.py	Cross-platform entry point: ui | campaign | report | targets | field; environment overrides.

Data flow

Stage	Where	Output
geometry gate	aero_auto.geometry_ok	rejects colliding/oversized/touching-ground stacks before JavaFoil
simulation	aero_auto.simulate	metrics dict per configuration (Cl/Cd/Cm, stall, peak)
campaign	aero_auto.campaign	dated run folder in the library: results.csv (+ per-height ground files)
analysis	aero_report / aero_targets	configs.csv, overview.xlsx, graphs, targets_report.csv
visualisation	flowfield.compute_field	field_<Run>.png velocity maps

2 Design decisions and rationale

This chapter answers "why does the code look the way it does?". Every decision below was made against a measured failure or a concrete constraint, not taste.

2.1 A calibrated field model instead of an in-house solver

The first implementation derived circulation from a constant-strength vortex-panel linear system whose boundary condition was zero normal velocity at collocation points. Releasing the TE Kutta condition in several arrangements (extra row, replacing the TE row, sources + one global circulation vortex at the TE then at the quarter chord, ridge/Tikhonov regularisation) never converged: residuals ~193 and absurd Cl (48) or negative Cl for cambered airfoils (-10.2 on s1223), plus a flat plate that still carried Cl 4.3 at zero angle. The normal-BC matrix is near-singular for multi-element stacks, TE-adjacent panels swamp the Kutta sensitivity, and sign extraction of Q = u - i v was a constant source of error. The pragmatic answer: keep JavaFoil as the authority and drive the field with the circulation it already produces. See chapter 7.

> **Note.** Polar numbers are always the authoritative data. The field images are illustrative and labelled as such in every generated graph.

2.2 One source of truth with call-your-own fallbacks

Campaign CSVs got richer over time - the 2025-era root results.csv carries neither geometry columns nor forces, the ground-effect files carry percent geometry but no millimetres. Rather than force a re-campaign, the report layer rebuilds what is missing (ensure_mm_columns, ensure_force_columns) so any historical file produces a complete report today.

2.3 Forces are magnitudes, drag is a budget

Downforce_N is stored as |Cl| q A (a magnitude, matching the aero_auto force_columns convention) because the steering/clamping side of the team writes it as a positive number; sign flips (ground vs inverted airfoil) are a property of the polar graph, not of the force model. CD*A is a roof: a budget shared with the rest of the car.

2.4 Tk is not thread-safe - so nothing touches it off the main thread

Running a JavaFoil analysis inside the Tk event loop freezes the window for 5-40 s. Running widgets from a worker thread crashes Tcl/Tk (observed with StringVar.get and widget.after directly). Resolution: workers only push messages to a queue.Queue; the main loop drains it on a 50 ms timer, so the UI is never mutated off the Tk thread (see 8.3).

2.5 Clock pinning and Windows portability

JavaFoil checks its release date at startup, so Linux runs wrap java in faketime pinned to @2020-01-01. Windows has no faketime; simulate now probes shutil.which('faketime') and falls back to a direct java call (best-effort - keep the machine clock sane, or run under WSL).

3 aero_auto.py - the engine

Everything below was added on top of the original single-element planner, in priority order: regulations first, geometry bookkeeping second, then the Windows-neutral runner and the force model.

CONSTANTS

Run surface & regulatory constants

Why. JavaFoil is sensitive to RE and the alpha sweep; the regulatory numbers come from FSUK 2026 / FSG v1.1 tables.

How. Module-level defaults keep the CLI questions short; the 120 s timeout protects against a hung java process; ELEMENT_WEIGHTS steer the campaign's exploration.

Used for: set once, overridable from the GUI and the CLI flags.

```
RE = 570000               # Reynolds number (replaced if you give chord+speed)
ALPHA = (-20, 5, 1)       # polar start, end, step (deg)
RUN_TIMEOUT = 120         # seconds per JavaFoil run before it is killed
MIN_CLEARANCE_PCT = 1.0   # dimensionless ground-clearance gate
ELEMENT_WEIGHTS = {2: 0.25, 3: 0.45, 4: 0.30}
FW_MAX_STACK_HEIGHT_MM = 250    FW_MAX_STACK_LENGTH_MM = 600
RW_MAX_TOTAL_HEIGHT_MM = 1100    RW_MAX_STACK_LENGTH_MM = 250
MIN_GROUND_CLEARANCE_MM = 35
```

```
aero_auto.py - default_reg() (the rule table)
def default_reg():
    """The standard FSUK 2026 / FSG v1.1 regulatory limits (mm)."""
    return dict(min_ground_clearance_mm=MIN_GROUND_CLEARANCE_MM,
                fw_max_stack_height_mm=FW_MAX_STACK_HEIGHT_MM,
                fw_max_stack_length_mm=FW_MAX_STACK_LENGTH_MM,
                rw_max_total_height_mm=RW_MAX_TOTAL_HEIGHT_MM,
                rw_max_stack_length_mm=RW_MAX_STACK_LENGTH_MM)
```

GEOMETRY

transform_coords / outlines_intersect / build_outlines

Why. Element placement is defined in percent-of-main-chord so any chord in mm can be applied later; intersection tests reject colliding stacks cheaply.

How. Each outline is resampled, rotated, scaled and translated from its unit-chord DAT profile; build_outlines optionally lifts the stack by the ground height so the geometry is available both translated (for thickness rules) and untranslated (rotation-invariant clash checks).

Used for: all millimetre columns, gap/overlap numbers, field images and GUI preview.

aero_auto.py - build_outlines()

```
def build_outlines(cfg, ground_h=UNSET):
    """Outlines of all elements. In ground effect every element is translated up by the wing height."""
    h = GROUND_H if ground_h is UNSET else ground_h
    if h is not None:
        h = validate_height(h)
    main, flaps = cfg
    if main not in AIRFOILS:
        raise KeyError(f"unknown main airfoil: {main!r}")
    outs = [AIRFOILS[main]]
    for (name, s, a, x, y) in flaps:
        if not name or name not in AIRFOILS:
            raise KeyError(f"unknown airfoil: {name!r}")
        outs.append(transform_coords(AIRFOILS[name], s, a, x, y))
    if h is not None:
        outs = [o + np.array([0.0, h / 100.0]) for o in outs]
    return outs
```

REGULATION

regulatory_reject / geometry_ok

Why. Offending stacks must not reach JavaFoil at all - feedback should be instant, not 5 s after a failed solve.

How. geometry_ok chains the cheap checks (neighbour clash heuristic, polygon intersection, dimensionless ground clearance) and then defers to regulatory_reject, which returns the first FSUK/FSG rule that is violated so the GUI can print exactly which limit broke.

Used for: the green/red PASS verdict in the GUI preview and the campaign's DONE gate.

aero_auto.py - geometry_ok()

```
def geometry_ok(cfg):
    """Cheap rejection of configs that would clash, intersect or touch the ground, or that break the
    FSUK/FSG size limits (never sent to JavaFoil)."""
    flaps = cfg[1]
    for k in range(len(flaps) - 1):
        _, s, a, x, y = flaps[k]
        if x + 0.8 * s >= flaps[k + 1][3]:
            return False
    outs = build_outlines(cfg, ground_h=None)          # untranslated: clashes are translation invariant
    for i, j in itertools.combinations(range(len(outs)), 2):
        if outlines_intersect(outs[i], outs[j]):
            return False
    mount = wing_mount_chord()
    ymin = min(o[:, 1].min() for o in outs)
    if GROUND_H is not None and mount is not None and (mount + ymin) * 100 < MIN_CLEARANCE_PCT:
        return False
    if regulatory_reject(outs, mount) is not None:
        return False
    return True
```

AUDIT

geometry_columns

Why. The team needs the numbers behind a verdict, comparable across runs.

How. Computes per-slot-pair gap/overlap/step and stack length/height/clearance in both % chord and millimetres once a chord is configured.

Used for: columns written into every result row and reused by the report/regression tests.

aero_auto.py - geometry_columns()

```
def geometry_columns(cfg, ground_h=UNSET):
```

```
    """Slot gap, overlap and height step between neighbouring elements, stack size, ground clearance.
    All in % of the main chord. Overlap > 0: the next element's leading edge is ahead of the previous
    trailing edge; overlap < 0: there is an open gap in x."""
    outs = build_outlines(cfg, ground_h)
    cols = {}
    for k in range(len(outs) - 1):
        p, q = outs[k], outs[k + 1]
        te = p[np.argmax(p[:, 0])]
        le = q[np.argmin(q[:, 0])]
        cols[f"{PAIRS[k]}_Gap_pct"] = 100 * _min_gap(p, q)
        cols[f"{PAIRS[k]}_Overlap_pct"] = 100 * (te[0] - le[0])
        cols[f"{PAIRS[k]}_Step_pct"] = 100 * (le[1] - te[1])
    allp = np.vstack(outs)
    ymin, ymax = allp[:, 1].min(), allp[:, 1].max()
    cols["Stack_Length_pct"] = 100 * (allp[:, 0].max() - allp[:, 0].min())
    cols["Stack_Height_pct"] = 100 * (ymax - ymin)
    cols["Stack_Ymin_pct"] = 100 * ymin
    cols["Stack_Ymax_pct"] = 100 * ymax
    h = GROUND_H if ground_h is UNSET else ground_h
    if h is not None:
        cols["Min_Clearance_pct"] = 100 * ymin
    if CHORD_MM:
        cols["Stack_Length_mm"] = (allp[:, 0].max() - allp[:, 0].min()) * CHORD_MM
        cols["Stack_Height_mm"] = (ymax - ymin) * CHORD_MM
        if h is not None:
            cols["Min_Clearance_mm"] = ymin * CHORD_MM
    return cols
```

FORCES

force_columns

Why. Dimensionless polars say nothing about how much downforce the car actually makes.

How. Downforce_N = |Cl_a0| q A and Drag_N = Cd_a0 q A with q = 0.5 * 1.225 * v^2 and A from chord * role span (fw 1400 mm / rw 950 mm).

Used for: every report table, the target report and the GUI result line.

aero_auto.py - force_columns()

```
def force_columns(m):
    """Downforce / drag in newtons at the design alpha. Assumes Cl and Cd are referenced to the main plane chord
    (check this in JavaFoil) and that the wing is a 2D extrusion over the given span, so it is an estimate."""
    if m is None:
        return {}
    chord_mm = speed = span_mm = None
    if PHYS and PHYS.get("span_mm") and PHYS.get("chord_mm") and PHYS.get("speed"):
        chord_mm, speed, span_mm = PHYS["chord_mm"], PHYS["speed"], PHYS["span_mm"]
    else:
        speed = SPEED
        chord_mm = CHORD_MM or DEFAULT_CHORD_MM
        span_mm = {"fw": DEFAULT_SPAN_FW_MM, "rw": DEFAULT_SPAN_RW_MM}.get(ROLE, 1000.0)
    if not (chord_mm and speed and span_mm):
        return {}
    cl0, cd0 = m.get("Cl_a0"), m.get("Cd_a0")
    if pd.isna(cl0) or pd.isna(cd0):
        return {}
    q = 0.5 * RHO * speed ** 2
    area = chord_mm / 1000 * span_mm / 1000
    return {"Downforce_N": abs(cl0) * q * area, "Drag_N": cd0 * q * area}
```

3.1 The JavaFoil runner

SIMULATE

simulate()

Why. A single, scriptable path was needed to analyse any stack reproducibly on both Linux and Windows.

How. One wing file is written per run (Automated Multi-Element Wing header, 9999.9 9999.9 element separators, 6-decimal coordinate lines), a JFS script drives Polar.Analyze over the RE/alpha surface, and java runs with faketime when present or bare otherwise. A per-run worker directory, its own session and a

timeout keep a hang from killing the campaign.

Used for: all polars flowing into results.csv.

aero_auto.py - simulate() - cmd construction incl. faketime fallback

```
def simulate(cfg, wdir, ground_h):
    """Runs one JavaFoil analysis inside the worker folder. Returns (metrics, status, debug lines)."""
    # Ensure airfoils are loaded in worker process
    if not AIRFOILS:
        AIRFOILS.update(load_all(AIRFOIL_DIR))
        if os.path.isdir(AIRFOIL_STORE):
            AIRFOILS.update(load_all(AIRFOIL_STORE))
    uid = uuid.uuid4().hex[:8]
    wing_fn, script_fn, polar_fn = f"wing_{uid}.txt", f"script_{uid}.jfs", f"polar_{uid}.txt"
    wing, script, polar = (os.path.join(wdir, f) for f in (wing_fn, script_fn, polar_fn))
    logf = os.path.join(wdir, f"java_{uid}.log")
    try:
        try:
            with open(wing, "w") as f:
                f.write("Automated Multi-Element Wing\n")
                for i, o in enumerate(build_outlines(cfg, ground_h)):
                    if i > 0:
                        f.write("9999.9 9999.9\n")
                    for pt in o:
                        f.write(f"{pt[0]:.6f} {pt[1]:.6f}\n")
            a0, a1, st = ALPHA
            with open(script, "w") as f:
                f.write(f"""Options.Country(10);
Geometry.Open("{wing_fn}");
Options.MachNumber(0);
Options.StallModel(0);
Options.TransitionModel(1);
Options.GroundEffect({1 if ground_h is not None else 0});
Options.HeightOverSpan(0.5);
Options.AspectRatio(0);
Options.SweepAngle(0.0);
Polar.Analyze({RE:g},{RE:g},{RE:g},{a0:g},{a1:g},{st:g},100,100,0,0);
Polar.Save("{polar_fn}");
JavaFoil.Exit();
""")
        except Exception as e:
            # A geometry or disk problem (bad config, full disk, permission) becomes a normal failed
            # run instead of an unhandled worker exception that aborts the batch.
            return (None, f"Setup Error: {type(e).__name__}: {e}", [f"cwd: {wdir}", str(e)])
        if shutil.which("faketime"):
            cmd = ["faketime", "-f", safe_date_str, "java", "-client", "-Xverify:none", "-XX:TieredStopAtLevel=1",
                   "-jar", JAVAFOIL_JAR, f"Script={script_fn}"]
        else:
            # Windows / minimal hosts: no faketime, so run Java directly (the date is only pinned for
            # JavaFoil's license check; one best-effort attempt).
            cmd = ["java", "-client", "-Xverify:none", "-XX:TieredStopAtLevel=1",
                   "-jar", JAVAFOIL_JAR, f"Script={script_fn}"]
        timed_out = False
        env = os.environ.copy()
        if os.name != "nt":
            env["DISPLAY"] = os.environ.get("DISPLAY", ":99")
        with open(logf, "w") as jl:
            try:
                proc = subprocess.run(cmd, cwd=wdir, stdout=jl, stderr=subprocess.STDOUT,
                                      timeout=RUN_TIMEOUT, env=env)
            except subprocess.TimeoutExpired as e:
                timed_out = True
                proc = e
        diag = [f"cwd: {wdir}", f"exit code: {getattr(proc, 'returncode', -1)}"]
        try:
            diag += [l.rstrip() for l in open(logf, errors="ignore").read().splitlines() if l.strip()][:8]
        except OSError:
            pass
        if timed_out:
            result = (None, "JavaFoil Timeout", diag)
        elif not (os.path.exists(polar) and os.path.getsize(polar) > 0):
            result = (None, "No Output File", diag)
```

```
        else:
            try:
                result = polar_metrics(polar)
            except Exception as e:
                result = (None, f"Parse Error: {e}", diag)
        if result[1] != "Valid":
            save_failure(uid, wdir, (wing, script, polar, logf), cmd)
        return result
    finally:
        for f in (wing, script, polar, logf):
            if os.path.exists(f):
                try: os.remove(f)
                except OSError: pass
```

PARSE

parse_polar / polar_metrics

Why. JavaFoil's .txt export needs tolerant parsing (header rows, unit rows).

How. parse_polar returns a frame plus the machine-readable headline; polar_metrics extracts Cl_peak, Cd_peak, Stall_Angle, Max_LD, Cl/Cd/Cm at the three design alphas (-2, 0, +2)

Used for: the FIELDS written into each row and consumed by reports and the GUI.

aero_auto.py - parse_polar()

```
def parse_polar(path):
    lines = [l for l in read_text(path).splitlines() if l.strip()]
    head = lines[:15]
    hdr = next((i for i, l in enumerate(lines)
                if re.search(r"alpha|α", l, re.I) and re.search(r"\bcl\b", l, re.I)), -1)
    if hdr == -1:
        return None, head

    def split(l):
        if "\t" in l: return [t.strip() for t in l.split("\t") if t.strip()]
        if ";" in l:  return [t.strip() for t in l.split(";") if t.strip()]
        return l.split()

    cols = [c.lower() for c in split(lines[hdr])]
    cols = ["alpha" if c in ("α", "a", "alpha") else c for c in cols]
    rows = []
    for l in lines[hdr + 1:]:
        toks = split(l)
        if len(toks) < len(cols):
            continue
        try:
            rows.append([float(t.strip().replace(",", ".")) for t in toks[:len(cols)]])
        except ValueError:
            continue
    if not rows:
        return None, head
    return pd.DataFrame(rows, columns=cols), head
```

3.2 The campaign planner

PLANNER

campaign() - three phases

Why. A brute-force sweep of the design space is far too large; a deadline-driven search fits a fixed pit session.

How. Phase 0 runs every main profile alone. Phase 1 (60% of the budget) samples random 2-4 element stacks weighted per ELEMENT_WEIGHTS; Phase 2 mutates the leaders via neighbours() until the deadline. Configs are deduplicated by cfg_key so nothing is solved twice.

Used for: autonomous exploration during pit changes, in-garage at home, and CI smoke runs.

aero_auto.py - campaign()

```
def campaign(hours, mains, flaps_pool):
    if WORKER_DIRS.empty():                          # direct callers (GUI/tools) may not have set these up
        setup_workers(WORKERS)
```

```
    deadline = time.time() + hours * 3600
    explore_end = time.time() + hours * 3600 * EXPLORE_FRACTION
    last_out = time.time()
    batch = max(WORKERS * 3, 6)

    # --- Phase 0: every airfoil alone ---
    singles = [(m, ()) for m in mains if cfg_key((m, ())) not in DONE]
    if singles:
        print(f"Phase 0: {len(singles)} single-element runs")
        run_batch(singles, "single", deadline)
        show(progress_line(force=True))

    # --- Phase 1: explore ---
    print("Phase 1: exploration")
    ns, ws = zip(*ELEMENT_WEIGHTS.items())
    while time.time() < explore_end and not STOP.is_set():
        cfgs = []
        while len(cfgs) < batch:
            n = random.choices(ns, weights=ws)[0]
            c = sample_config(n, mains, flaps_pool)
            if c is not None and cfg_key(c) not in {cfg_key(x) for x in cfgs}:
                cfgs.append(c)
            elif c is None:
                break
        if not cfgs:
            print("No new valid configs left to sample.")
            break
        run_batch(cfgs, "explore", explore_end)
        show(progress_line())
        check_fail_streak()
        if time.time() - last_out > SUMMARY_EVERY:
            write_outputs(); last_out = time.time()

    # --- Phase 2: refine ---
    print("Phase 2: refinement")
    k = TOP_K
    cap = MAX_ROUND or WORKERS * 20
    while time.time() < deadline and not STOP.is_set():
        round_cfgs, seen = [], set()
        for n in (2, 3, 4):
            for p in best_parents(n, k):
                for c in neighbours(p, mains, flaps_pool):
                    key = cfg_key(c)
                    with LOCK:
                        known = key in DONE
                    if not known and key not in seen and geometry_ok(c):
                        seen.add(key); round_cfgs.append(c)
        random.shuffle(round_cfgs)
        round_cfgs = round_cfgs[:cap]
        if not round_cfgs:
            if k < 60:
                k += 8
                print(f"Neighbourhood exhausted, widening to the best {k} per element count")
                continue
            print("Refinement converged (nothing new to test).")
            break
        run_batch(round_cfgs, "refine", deadline)
        show(progress_line())
        check_fail_streak()
        if time.time() - last_out > SUMMARY_EVERY:
            write_outputs(); last_out = time.time()
```

SCORING

rate() + run_one()

Why. The search needs a single scalar that rewards lift while punishing drag so it does not chase oversized stacks.

How. rate converts the metrics into a 0-100 score (uplift dominant, drag as a discount); run_one wraps the simulate->rate pipeline and appends the enriched row under a lock, tagging it into DONE.

Used for: leader selection, Tier/Score in every results file.

```
aero_auto.py - rate()
def rate(m, status):
    """Returns (tier, score). Tier 4 = rejected."""
    if status != "Valid" or m is None:
        return 4, float("nan")
    cl0, cd0, st = m["Cl_a0"], m["Cd_a0"], m["Stall_Angle"]
    if any(pd.isna(v) for v in (cl0, cd0, st)) or cl0 >= 0 or cd0 <= 0 or cd0 > CD_MAX or st <= ALPHA[0]:
        return 4, float("nan")
    margin = DESIGN_ALPHA - st
    tier = 1 if margin >= TIER1_MARGIN else 2 if margin >= TIER2_MARGIN else 3
    df_s = clip01((abs(cl0) - DF_LO) / (DF_HI - DF_LO))
    mg_s = clip01(margin / MARGIN_FULL)
    ef_s = clip01((abs(cl0) / cd0 - LD_LO) / (LD_HI - LD_LO))
    lo = min(abs(m["Cl_am2"]), abs(m["Cl_ap2"])) if pd.notna(m["Cl_am2"]) and pd.notna(m["Cl_ap2"]) else float("nan")
    rb_s = clip01((lo / abs(cl0) - 0.8) / 0.2) if pd.notna(lo) else 0.5
    return tier, 100 * (W_DF * df_s + W_MG * mg_s + W_EF * ef_s + W_RB * rb_s)
```

OUTPUT

write_row / fmt_value

Why. CSV columns must stay stable across versions even if the dict grows.

How. A canonical FIELDS order is enforced at write time with per-column number formatting; extrasaction=ignore keeps new diagnostics from breaking older readers. Every row is stamped with a wall-clock Date (ISO, seconds), the Duration_s it took to compute and the Alpha_Interval + RE it was run at, so a results file doubles as the log of how many simulations were done, when, and the total time invested.

Used for: results.csv in the campaign output folder.

```
aero_auto.py - write_row()
def write_row(row):
    path = RESULTS_FILE[0]
    new = not os.path.exists(path)
    if not new and results_header(path) != FIELDS:
        # The file on disk has an older/wider header than FIELDS; appending FIELDS rows now would
        # misalign every line. Migrate it to the current layout once, then append normally.
        normalize_results_file(path)
    with open(path, "a", newline="") as f:
        w = csv.DictWriter(f, fieldnames=FIELDS, extrasaction="ignore")
        if new:
            w.writeheader()
        w.writerow({k: fmt_value(k, row.get(k)) for k in FIELDS})
```

4 aero_targets.py - the target report

Design goals are set as (CL*A target, CD*A roof). The guide-targets tool compares every valid row and prints the deviation against the ±0.4 tolerance band.

TARGETS

evaluate()

Why. After a campaign the team needs to know which stacks hit the design point, not just which is fastest.

How. Computes CL*A and CD*A with the role span and the row chord, then scores the deviation from the target and flags how much drag budget remains.

Used for: targets_report.csv plus the ranked console list.

```
aero_targets.py - evaluate()
def evaluate(d, area, tol, tgt_cl_a, tgt_cd_a, tgt_ld):
    d = d.copy()
    d["Area_m2"] = area
    d["CL_A_actual"] = d["Cl_a0"].abs() * area
    d["CD_A_actual"] = d["Cd_a0"] * area
    d["LD_actual"] = d["CL_A_actual"] / d["CD_A_actual"].replace(0, np.nan)
    d["Dev_CL_A"] = d["CL_A_actual"] - tgt_cl_a
    d["Dev_CD_A"] = d["CD_A_actual"] - tgt_cd_a
    d["Dev_LD"] = d["LD_actual"] - tgt_ld
```

```
    d["Within_Tol"] = d["Dev_CL_A"].abs() <= tol
    d["CD_A_under_roof"] = d["CD_A_actual"] <= tgt_cd_a
    band = d[["Cl_am2", "Cl_a0", "Cl_ap2"]].abs()
    d["Band_MinAbs"] = band.min(axis=1, skipna=True)
    d["Band_Swing"] = (band.max(axis=1) - band.min(axis=1)).where(
        band.notna().all(axis=1), np.nan)
    d["Dev_Sum"] = d["Dev_CL_A"].abs() + 0.1 * d["Dev_CD_A"].abs()
    return d
```

REPORT

sorted_report()

Why. The report must be stable and direction-consistent (closer to CL*A first, lower CD*A when equal).

How. Sorts on |deviation| then CD*A ascending, keeping the leader compact.

Used for: the ordered table exported next to the results file.

```
aero_targets.py - sorted_report()
def sorted_report(d):
    d = d.sort_values(["Within_Tol", "Dev_Sum", "Band_Swing", "Score"],
                      ascending=[False, True, True, False]).dropna(subset=["Cl_a0"])
    d.insert(0, "Rank", range(1, len(d) + 1))
    out_cols = ["Rank", "Within_Tol", "Area_m2", "CL_A_actual", "CD_A_actual", "LD_actual",
                "Dev_CL_A", "Dev_CD_A", "Dev_LD", "CD_A_under_roof", "Band_MinAbs", "Band_Swing"] + A.FIELDS
    return d.reindex(columns=[c for c in out_cols if c in d.columns])
```

5 flowfield.py - the schematic potential-flow model

The field model is deliberately not a force solver. It produces a believable velocity map whose circulation comes from the config's own polar Cl, so the picture always agrees with the numbers.

```
Q(z)  =  sum over source panels            (geometry blocking, block = 0.18 per panel)
      +  sink at each 1/4-chord point      (cancels the element's net source flux)
      +  point vortex at each 1/4 chord    Gamma_e = -Cl_e * U * c_e / 2
      +  ground image terms                mirror at y = 0;
                                            sources/sinks keep sign, vortices flip sign

softening: 1/(z-zk)  ->  conj(z-zk) / (|z-zk|^2 + rc^2)      rc = 1e-2
field interior: points inside any element outline are NaN (meshed away)
```

PANELS

build_panels / _orient_out

Why. Continuous source sheets are cheaper and smoother than many point sources, and the orientation drives the outward-normal sign convention.

How. Each outline is oriented counter-clockwise, then subdivided into short panels (max spacing 0.008 chords) whose strengths are scaled by the panel length * block factor.

Used for: the blocking term that stops the freestream from leaking through the wings.

```
flowfield.py - build_panels()
def build_panels(outs, max_spacing=0.008):
    """Panel list {p1, p2, m, s, t, n, el} for each element outline, cw, outward normals."""
    panels = []
    starts = []
    decs = []
    centroid = np.zeros(2)
    total = sum(len(o) for o in outs)
    for o in outs:
        centroid += o.sum(axis=0)
    centroid /= max(total, 1)
    for ei, o0 in enumerate(outs):
        o = _orient_out(o0)
        if max_spacing > 0:
            cum = np.concatenate([[0], np.cumsum(np.hypot(np.diff(o[:, 0]), np.diff(o[:, 1])))])
            want = max(8, int(cum[-1] / max_spacing))
            if want < len(o):
                idx = np.unique(np.linspace(0, len(o) - 1, want).astype(int))
                o = o[idx]
```

```
        starts.append(len(panels))
        decs.append(o)
        n = len(o)
        for i in range(n):
            p1 = o[i]
            p2 = o[(i + 1) % n]
            m = (p1 + p2) / 2.0
            d = p2 - p1
            s = np.hypot(d[0], d[1])
            if s < 1e-12:
                continue
            t = d / s
            nvec = np.array([t[1], -t[0]])
            if np.dot(nvec, m - centroid) < 0:
                nvec = -nvec
            panels.append(dict(p1=p1, p2=p2, m=m, s=s, t=t, n=nvec, el=ei))
    return panels, np.array(starts), decs
```

KERNELS

_soft / _vel_source / _vel_sink_pt / _vel_vortex_pt

Why. Raw point potentials blow up to |V| ~ 30x freestream at the singular cores; the softening radius caps them so the colour map stays readable.

How. Each kernel is the standard complex-potential velocity folded through the softening cap; the vortex sign matches the Cl convention verified in section 2.1.

Used for: the three building blocks of the field.

flowfield.py - _soft()

```
def _soft(z, rc):
    """Return z with any values inside the core radius rc pushed to its boundary."""
    m = np.abs(z)
    small = m < rc
    if np.any(small):
        z = np.where(small, rc * z / np.where(m == 0, 1.0, m), z)
    return z
```

CIRCULATION

element_circulation / element_quarter

Why. The vortices must encode the polar's lift at the alpha the user asked for.

How. Gamma_e = -Cl_e * U * c_e / 2; Cl_e is split across the stack's elements; each vortex sits at its element's quarter chord.

Used for: makes the image agree with the p o l a r at every plotted run.

flowfield.py - element_circulation()

```
def element_circulation(outs, Cls, alpha):
    """Per element: normalised strength Gamma_e (uses Cl per element or a thin-airfoil guess)."""
    a = np.radians(alpha)
    G = np.zeros(len(outs))
    for ei, o in enumerate(outs):
        c = o[:, 0].max() - o[:, 0].min()
        cl = Cls[ei] if Cls is not None and ei < len(Cls) and Cls[ei] is not None else 2 * np.pi * np.sin(a)
        G[ei] = -cl * UINF * max(c, 1e-9) / 2.0
    return G
```

FIELD

compute_field() - chunked, vectorised

Why. A 300x300 four-element ground field must render in ~20-30 s, not minutes.

How. Source-panel velocities are summed over panel batches (chunk = 4096) against the full grid of points; the interior-mask is decided by _points_in_poly_many over all elements; ground images are folded in at the sensor level.

Used for: every field_<Run>.png and the render_field CLI.

flowfield.py - compute_field()

```
def compute_field(outs, alpha=0.0, ground_h=None, Cls=None, bounds=None, nx=240, ny=240,
                  block=0.18):
```

```
    """Illustrative velocity field. Returns X, Y, U, V (inside elements = NaN).

    Cls: per-element lift coefficients (in the aero_convention, so s1223 inverted is ~ -1.6).
    ground_h: distance from ground (y=0) to the leading edge. Enables the mirrored ground.
    """
    outs = [np.asarray(o, float) for o in outs]
    a = np.radians(alpha)
    vinf = np.array([np.cos(a), np.sin(a)]) * UINF
    panels, _starts, decs = build_panels(outs)
    Gamma = element_circulation(outs, Cls, alpha)
    qpts = [element_quarter(o) for o in decs]
    z1a = np.array([p["p1"][0] + 1j * p["p1"][1] for p in panels])
    z2a = np.array([p["p2"][0] + 1j * p["p2"][1] for p in panels])
    sa = np.array([p["s"] for p in panels])
    qx = np.array([q[0] + 1j * q[1] for q in qpts])
    mivec = np.array([block * UINF * float(sum(p["s"] for p in panels if p["el"] == ei))
                      for ei in range(len(outs))])
    mirror_on = ground_h is not None

    if bounds is None:
        xmin = min(o[:, 0].min() for o in outs)
        xmax = max(o[:, 0].max() for o in outs)
        ymin = min(o[:, 1].min() for o in outs)
        ymax = max(o[:, 1].max() for o in outs)
        span = max(xmax - xmin, ymax - ymin, 0.5)
        bounds = (xmin - 0.35 * span, xmax + 0.45 * span, ymin - 0.7 * span, ymax + 0.7 * span)
    x, y = np.meshgrid(np.linspace(bounds[0], bounds[1], nx), np.linspace(bounds[2], bounds[3], ny))
    xf = x.ravel()
    yf = y.ravel()
    z = xf + 1j * yf

    u = np.full(z.shape, vinf[0])
    v = np.full(z.shape, vinf[1])
    chunk = 4096
    for i in range(0, len(z), chunk):
        zc = z[i:i + chunk]
        dz1 = _soft(zc[:, None] - z1a[None, :], 1e-2)
        dz2 = _soft(zc[:, None] - z2a[None, :], 1e-2)
        src = (np.log(dz1) - np.log(dz2)) / (2 * np.pi)
        vel = (src * sa[None, :] * (block * UINF)).sum(axis=1)
        if mirror_on:
            zd = zc.real[:, None] + 1j * (-zc.imag[:, None])
            mdz1 = _soft(zd - z1a[None, :], 1e-2)
            mdz2 = _soft(zd - z2a[None, :], 1e-2)
            msrc = (np.log(mdz1) - np.log(mdz2)) / (2 * np.pi)
            vel = vel + (msrc * sa[None, :] * (block * UINF)).sum(axis=1)
        # sinks + circulation vortices, and their ground images
        for ei in range(len(outs)):
            vel = vel + _vel_sink_pt(zc, qx[ei], mivec[ei])
            vel = vel + _vel_vortex_pt(zc, qx[ei], Gamma[ei])
            if mirror_on:
                vel = vel + _vel_sink_pt(zc, qx[ei].conjugate(), mivec[ei])
                vel = vel + _vel_vortex_pt(zc, qx[ei].conjugate(), -Gamma[ei])
        u[i:i + chunk] = u[i:i + chunk] + np.real(vel)
        v[i:i + chunk] = v[i:i + chunk] + (-np.imag(vel))

    inside = _points_in_poly_many(outs, xf, yf)
    u[inside] = np.nan
    v[inside] = np.nan
    return x, y, u.reshape(x.shape), v.reshape(x.shape)
```

TRACES

streamlines

Why. Streamlines make separation and the ground jet readable in one glance.

How. Uni-directional tracing from a list of seeds, integrated in normalised data coordinates and stopped at NaN or near-zero speed.

Used for: the white trace overlay on the field images.

flowfield.py - streamlines()

```
def streamlines(x, y, u, v, seeds, n=600, ds=None):
    """Trace streamlines from seeds (list of (x0, y0)) both up- and down-flow. Returns list of
    point lists in data coordinates."""
    if ds is None:
        ds = (x[0, -1] - x[0, 0]) / (u.shape[1] * 4)
    ok = np.isfinite(u)
    uu = np.where(ok, u, 0.0)
    vv = np.where(ok, v, 0.0)
    x0 = x[0, 0]
    y0 = y[0, 0]
    hx = (x[0, 1] - x[0, 0])
    hy = (y[1, 0] - y[0, 0])
    nx_, ny_ = u.shape
    lines = []

    def step(cur, sx0, sy0, dirn):
        c = np.array([sx0, sy0], float)
        out = []
        for _ in range(n):
            ix = int((c[0] - x0) / hx)
            iy = int((c[1] - y0) / hy)
            if not (0 <= ix < nx_ and 0 <= iy < ny_):
                break
            s = float(np.hypot(uu[iy, ix], vv[iy, ix]))
            if s < 1e-5 or not ok[iy, ix]:
                break
            c = c + dirn * np.array([uu[iy, ix], vv[iy, ix]]) / s * ds
            out.append((float(c[0]), float(c[1])))
        return out

    for (sx0, sy0) in seeds:
        up = step(0, sx0, sy0, +1)
        down = step(0, sx0, sy0, -1)
        pts = list(reversed(down)) + [(sx0, sy0)] + list(up)
        lines.append(pts)
    return lines
```

5.1 Validation invariants

The model was checked against physical invariants during development. Numbers are from the 240x240 self-test field.

Check	Result
ground impermeability, max |v|/U	< 0.15, confined to a narrow near-wall strip (wall artifacts)
far-field freestream, corners	within ±10% of U
minimum speed (stagnation-like)	~ 0.01 at the leading-edge cores
max speed	~14-15, localised at softened singular cores (clipped at display)
interior masking	NaN inside element polygons, meshed out of the colour map

6 aero_report.py - reporting pipeline

The report layer turns any results CSV into a folder of ranked tables, an Excel workbook, a legend, and graphs - without requiring a re-campaign.

LOAD

load() + quality_rank()

Why. Reports must survive messy historical CSVs and sort on the same criteria the campaign used.

How. Filters Tier<=3/Valid/dropNa, keeps only the requested airfoils, then sorts on Score, Max_LD, Downforce_N desc and Drag_N asc (only for columns that exist) and stamps a Rank.

Used for: the ranked view used everywhere downstream.

aero_report.py - quality_rank()

```
def quality_rank(d):
    d = d.copy()
    order = ("Score", "Max_LD", "Downforce_N", "Drag_N")
    cols = [c for c in order if c in d.columns]
    asc = [c == "Drag_N" for c in cols]
    d = d.sort_values(cols, ascending=asc) if cols else d.sort_index()
    d["Rank"] = np.arange(1, len(d) + 1)
    return d
```

SELECT

select_top()

Why. A naive top-N would be wall-to-wall four-element stacks; the report should show the best of every family.

How. Round-robins across the distinct Elements counts, then ranks each bucket with quality_rank.

Used for: Best/configs.csv.

aero_report.py - select_top()

```
def select_top(d, n):
    """Round-robin across element counts so the top list contains every stack type if possible."""
    if "Elements" not in d.columns or n < 2 or len(d) <= n:
        return quality_rank(d).head(n)
    d = d.copy()
    groups = [g for _, g in d.groupby("Elements") if not g.empty]
    picked = []
    i = 0
    while len(picked) < n and groups:
        for g in groups:
            if len(picked) >= n:
                break
            if i < len(g):
                picked.append(g.iloc[i])
        i += 1
        groups = [g for g in groups if i < len(g)]
    top = pd.DataFrame(picked)
    return quality_rank(pd.concat([top, d.drop(top.index)]))[:n]
```

REPAIR

ensure_mm_columns / ensure_force_columns

Why. Older result files (2025-era root, ground-effect files) lack millimetre geometry or forces.

How. Rebuilds the mm columns from build_outlines * chord, and the force columns from q = 0.5 rho v^2 with the role span; only for missing columns so fresh files are untouched.

Used for: full reports from any historical results file.

aero_report.py - ensure_mm_columns()

```
def ensure_mm_columns(d, chord):
    """results.csv from older campaigns may lack the mm geometry columns; rebuild them."""
    d = d.copy()
    needs = [c for c in ("Stack_Length_mm", "Stack_Height_mm", "Min_Clearance_mm") if c not in d.columns]
    if not needs:
        return d
    for i in range(len(d)):
        height = None if str(d.iloc[i].get("Flow", "")).lower() != "ground" \
            else float(d.iloc[i].get("Height_pct", 0) or 0)
        cfg = A.row_to_cfg(d.iloc[i])
        outs = A.build_outlines(cfg, ground_h=height)
        allp = np.vstack(outs) * chord
        vals = {"Stack_Length_mm": float(allp[:, 0].max() - allp[:, 0].min()),
                "Stack_Height_mm": float(allp[:, 1].max() - allp[:, 1].min())}
        if height is not None:
            vals["Min_Clearance_mm"] = float(allp[:, 1].min())
        for c in needs:
            d.at[d.index[i], c] = vals.get(c, np.nan)
    return d
```

NAMES

rename_columns() - duplicate-safe human names

Why. Both Stack_Length_pct and Stack_Length_mm read as "Stack length"; found as duplicate headers in generated CSVs.

How. Renames via the LEGEND table and, when a human title repeats, appends the unit in parentheses (Stack length vs Stack length (mm)).

Used for: clean, self-explanatory results_human.csv headers.

```
aero_report.py - rename_columns()
def rename_columns(d):
    renamed = {}
    for c in d.columns:
        name = LEGEND[c][0] if c in LEGEND else human_name(c)
        if name in renamed.values():
            unit = LEGEND[c][1] if c in LEGEND else ""
            name = f"{name} ({unit})" if unit else f"{name}*"
        renamed[c] = name
    out = d.rename(columns=renamed)
    for c in ("Flow", "Elements", "Run", "Phase", "Main airfoil"):
        if c in out.columns:
            out[c] = out[c].astype(str)
    return out
```

LEGEND

write_legend() / write_xlsx()

Why. Every column must be explainable to someone who did not write the code; the workbook has to open in Excel/LibreOffice without pandas-specific types.

How. write_legend dumps the column/unit/description table and now intentionally includes Status and Rank; write_xlsx emits Top configs, All rows (chunked to 500 per block) and Legend sheets.

Used for: columns_legend.csv and Best/overview.xlsx.

```
aero_report.py - write_legend()
def write_legend(out_path):
    rows = []
    for col in COLUMNS:
        if col in LEGEND:
            title, unit, desc = LEGEND[col]
        else:
            title, unit, desc = human_name(col), "", ""
        rows.append({"column": col, "meaning": title, "unit": unit, "description": desc})
    pd.DataFrame(rows).to_csv(out_path, index=False)
```

GRAPHS

plot_polar / plot_geometry / plot_field

Why. Engineers want the three views together: the numbers, the placement, and the flow shape.

How. plot_polar fits a quadratic through Cl at -2/0/+2 deg and marks Cl_peak and stall; plot_geometry draws the stack in millimetres with gap/overlap and the ground line; plot_field reuses flowfield for the velocity map with masked wings, clipped speed and streamlines.

Used for: graphs/polar_<Run>.png, geometry_<Run>.png, field_<Run>.png.

```
aero_report.py - plot_polar()
def plot_polar(r, out_png):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    alphas = np.array([-2.0, 0.0, 2.0])
    cls = np.array([float(r.get(c, np.nan)) for c in ("Cl_am2", "Cl_a0", "Cl_ap2")])
    fig, ax = plt.subplots(figsize=(7.5, 5.2))
    if not np.isnan(cls).any():
        aa = np.linspace(-6, 6, 60)
        p = np.polyfit(alphas, cls, 2)
        ax.plot(aa, np.polyval(p, aa), color="#c8102e", lw=1.2, alpha=0.7, label="quadratic fit")
        ax.plot(alphas, cls, "o", color="#0b3d91", ms=6, label="JavaFoil points")
        for a_, c_ in zip(alphas, cls):
            ax.annotate(f"{c_:.3f}", (a_, c_), textcoords="offset points", xytext=(0, 7), fontsize=8, ha="center")
    pk = float(r.get("Cl_peak", np.nan))
```

```
    st = float(r.get("Stall_Angle", np.nan))
    if not np.isnan(pk):
        ax.plot([st], [pk], "v", color="k", ms=8, label=f"peak {pk:.3f} @ {st:+.0f} deg")
    ax.axhline(0, color="k", lw=0.6)
    ax.set_xlabel("alpha (deg)")
    ax.set_ylabel("Cl")
    ax.grid(alpha=0.3)
    cd = float(r.get("Cd_a0", np.nan))
    ld = float(r.get("Max_LD", np.nan))
    dn = float(r.get("Downforce_N", np.nan))
    extra = " | ".join(f"{k}: {float(v):.3g}" for k, v in
                       (("Cl@0", r.get("Cl_a0")), ("Cd@0", cd), ("|Cl|/Cd", ld)) if not np.isnan(float(v)))
    ax.set_title(f"Polar {r['Run_ID']} - {A.describe(A.row_to_cfg(r))}\n{extra}"
                 + (f"   Downforce {dn:.1f} N" if not np.isnan(dn) else ""))
    ax.legend(fontsize=8, loc="best")
    plt.tight_layout()
    plt.savefig(out_png, dpi=110)
    plt.close(fig)
```

RESIM

resim_polar_curve()

Why. Campaign polars use a coarse alpha step; the top few configs deserve a smooth, denser curve for the report.

How. Writes a fresh wing/JFS pair for the chosen config, runs Polar.Analyze over a finer sweep (default 4..26 step 2), parses and returns the denser frame, and cleans up its temp files. Uses the absolute jar path (the bare basename only worked inside campaign worker dirs).

Used for: the --resim report flag.

aero_report.py - resim_polar_curve()

```
def resim_polar_curve(r, wdir, chord, alphas=(-18, 20, 2)):
    """Re-run JavaFoil with a denser alpha sweep for one config. Returns (alpha, cl, cd) rows."""
    import uuid
    height = None if str(r.get("Flow", "")).lower() != "ground" else float(r.get("Height_pct", 0) or 0)
    cfg = A.row_to_cfg(r)
    os.makedirs(wdir, exist_ok=True)
    uid = uuid.uuid4().hex[:8]
    wing = os.path.join(wdir, f"wing_{uid}.txt")
    script = os.path.join(wdir, f"script_{uid}.jfs")
    polar = os.path.join(wdir, f"polar_{uid}.txt")
    logf = os.path.join(wdir, f"java_{uid}.log")
    with open(wing, "w") as f:
        f.write("Automated Multi-Element Wing\n")
        for i, o in enumerate(A.build_outlines(cfg, ground_h=height)):
            if i > 0:
                f.write("9999.9 9999.9\n")
            for pt in o:
                f.write(f"{pt[0]:.6f} {pt[1]:.6f}\n")
    a0, a1, st = alphas
    with open(script, "w") as f:
        f.write(f"""Options.Country(10);
Geometry.Open("{wing}");
Options.MachNumber(0);
Options.StallModel(0);
Options.TransitionModel(1);
Options.GroundEffect({1 if height is not None else 0});
Options.HeightOverSpan(0.5);
Options.AspectRatio(0);
Options.SweepAngle(0.0);
Polar.Analyze({A.RE:g},{A.RE:g},{A.RE:g},{a0:g},{a1:g},{st:g},100,100,0,0);
Polar.Save("{os.path.basename(polar)}");
JavaFoil.Exit();
""")
    cmd = ["faketime", "-f", A.safe_date_str, "java", "-client", "-Xverify:none", "-XX:TieredStopAtLevel=1",
           "-jar", os.path.join(A.JAVA_DIR, A.JAVAFOIL_JAR), f"Script={os.path.basename(script)}"]
    with open(logf, "w") as jl:
        proc = subprocess.Popen(cmd, cwd=wdir, stdout=jl, stderr=subprocess.STDOUT, start_new_session=True)
        try:
            proc.wait(timeout=3 * A.RUN_TIMEOUT)
        except subprocess.TimeoutExpired:
```

```
            proc.kill()
            proc.wait()
    for f in (wing, script, logf):
        try:
            os.remove(f)
        except OSError:
            pass
    if not (os.path.exists(polar) and os.path.getsize(polar) > 0):
        return None
    df, _ = A.parse_polar(polar)
    try:
        os.remove(polar)
    except OSError:
        pass
    if df is None or not {"alpha", "cl", "cd"} <= set(df.columns):
        return None
    return df.dropna(subset=["alpha", "cl", "cd"])
```

7 aero_ui.py - the threaded GUI

Six tabs wrap the engine - Configure, Airfoils, Campaigns, Results, Graphs and Optimizer - plus a menu bar, a status bar and a settings file that survives restarts. Every long-lived operation runs in a daemon thread; nothing touches Tk except the main loop.

APP

App.__init__ - queue-based thread bridge

Why. See 2.4: Tk cannot be driven off the main thread without corruption/crashes.

How. A queue.Queue plus a 50 ms main-loop drain (see 8.3) moves all widget updates to the Tk thread; the icon is resolved from the bundle (sys._MEIPASS) or the 'assets' folder next to the code.

Used for: all worker->UI traffic in the app.

```
aero_ui.py - App.__init__()
def __init__(self):
    super().__init__()
    # Read persisted state before any widget exists so the saved theme and
    # description settings are installed before the first widget is built.
    self._state = self._read_state()
    s = self._state.get("settings", {}) if isinstance(self._state, dict) else {}
    self._settings = {
        "theme": s.get("theme", DEFAULT_THEME),
        "tips": bool(s.get("tips", True)),
        "tip_delay": int(s.get("tip_delay", 450) or 0),
        "tip_wrap": int(s.get("tip_wrap", 420) or 420),
    }
    if self._settings["theme"] not in THEMES:
        self._settings["theme"] = DEFAULT_THEME
    self._pal = _apply_palette(self._settings["theme"])
    self._pal_name = self._settings["theme"]
    _TIP.set_options(enabled=self._settings["tips"],
                     delay=self._settings["tip_delay"],
                     wraplength=self._settings["tip_wrap"])
    self._setup_theme()
    self.title("aero_ui - multi-element wing campaign shell")
    self.geometry("1240x820")
    self._load_icon()
    A.AIRFOILS.update(A.load_all(A.AIRFOIL_DIR))
    if os.path.isdir(A.AIRFOIL_STORE):
        A.AIRFOILS.update(A.load_all(A.AIRFOIL_STORE))
    # ensure worker directories exist and are populated
    from queue import Queue
    import shutil
    A.WORKER_DIRS = Queue()
    w = os.path.join(os.path.dirname(os.path.abspath(__file__)), "workers")
    os.makedirs(w, exist_ok=True)
    for j in ("javafoil.jar", "mhclasses.jar"):
        src = os.path.join(A.JAVA_DIR, j)
```

```
        dst = os.path.join(w, j)
        if os.path.exists(src) and not os.path.exists(dst):
            shutil.copy(src, dst)
    A.WORKER_DIRS.put(w)
    self.names = sorted(A.AIRFOILS.keys())
    self.results = pd.DataFrame()
    self._tree_frame = pd.DataFrame()
    self.threads = []
    self.ui_q = queue.Queue()
    self._cli_lock = threading.Lock()   # serialises report/targets (they mutate global sys.argv)
    self.cam_t0 = None
    self.cam_runs_base = 0
    self.cam_active = False
    self._gimg = None
    self._graph_paths = []
    self._drain_timer_running = True
    self.gh_state = tk.StringVar(value="300")
    self.ge_state = tk.BooleanVar(value=True)
    self.custom_span_state = tk.StringVar(value="1000")
    self.gr_results = tk.StringVar(value=os.path.join(A.OUT_DIR, "results.csv"))
    self.after(50, self._drain_ui)
    self._build_ui()
    self._apply_state(self._state)
    self._refresh_runs()
    self.load_results_cache()
    self.refresh_graph_files()
    self.protocol("WM_DELETE_WINDOW", self._on_close)
```

ICON

_bundle_root / _load_icon

Why. The same source must work from the working tree, the installed ~/Applications copy, and the frozen PyInstaller exe.

How. Chooses sys._MEIPASS when frozen, otherwise the module directory; tries icon.ico, then icon.png, then the original logo, and finally an AERO_ICON env override - silently skipping whatever is absent.

Used for: window icon on every platform and build mode.

aero_ui.py - App._load_icon()

```
def _load_icon(self):
    root = self._bundle_root()
    candidates = [os.environ.get("AERO_ICON"),
                  os.path.join(root, "assets", "icon.ico"),
                  os.path.join(root, "assets", "icon.png"),
                  os.path.join(root, "assets", "Logoertblanc.png")]
    for p in candidates:
        if not p or not os.path.exists(p):
            continue
        try:
            if p.lower().endswith(".ico"):
                self.iconbitmap(p)
            else:
                self.iconphoto(True, tk.PhotoImage(file=p))
            return
        except Exception:
            continue
```

CHECK

on_check() / on_simulate()

Why. Instant geometry feedback and one-click real simulation are the two entry points everyone uses; both must be non-blocking.

How. on_check routes through geometry_ok + geometry_columns and redraws the embedded matplotlib preview; on_simulate reads all inputs on the main thread, then runs run_one in a worker and posts the result through the queue.

Used for: the Configure tab buttons.

aero_ui.py - App.on_simulate()

```
def on_simulate(self):
```

```
if getattr(self, "_sim_running", False):
    self.log("A simulation is already running - wait for it to finish.")
    return
cfg = self.read_cfg()
if cfg is None:
    return
try:
    re0 = float(self.vars["re"].get())
    chord0 = float(self.vars["chord"].get())
    speed0 = float(self.vars["speed"].get())
except ValueError:
    self.log("numeric chord/speed/RE required"); return
role0 = ATTR[self._role_state.get()][0]
reg0 = self.reg_state.get()
gh0 = None
if self.ge_state.get() and self.gh_state.get().strip() not in ("", "none", "None"):
    try:
        gh0 = float(self.gh_state.get())
    except ValueError:
        self.log("ground height must be a number"); return
    if gh0 <= 0:
        self.log("ground height must be positive (% of the main chord)"); return
if role0 == "custom":
    try:
        span0 = float(self.custom_span_state.get())
    except ValueError:
        span0 = 1000.0
else:
    span0 = ATTR[self._role_state.get()][1]
desc = self.main_box.get()

prog = tk.Toplevel(self)
prog.title("Simulating...")
prog.transient(self)
prog.resizable(False, False)
ttk.Label(prog, text=f"Running JavaFoil simulation...\n{desc} + flaps\nThis may take 10-30 seconds.").pack(padx=2
pb = ttk.Progressbar(prog, mode="indeterminate", length=300)
pb.pack(padx=20, pady=(0, 15))
pb.start(100)
self._sim_prog = prog
self._sim_running = True
try:
    run_dir, results_path = self._new_run(
        "single", desc, main=desc,
        flow=("ground" if gh0 is not None else "unbounded"),
        height_pct=gh0, n_runs=0)
except Exception as e:
    self._sim_running = False
    prog.destroy()
    self.log(f"could not create run folder: {e}"); return
self.log(f"simulating {desc} + flaps at alpha {A.ALPHA[0]}..{A.ALPHA[1]} "
         f"step {A.ALPHA[2]}, RE={re0:g}, "
         f"{'ground' if gh0 is not None else 'unbounded'} ...")

def work():
    try:
        A.RE = re0
        A.SPEED = speed0
        A.CHORD_MM = chord0
        A.ROLE = role0
        A.SPAN = span0
        A.REG = A.default_reg()
        A.REG_ENFORCE = reg0
        A.set_ground_h(gh0)
        A.RESULTS_FILE[0] = results_path
        row = A.run_one(cfg, "ui", gh0)
        A.write_run_meta(run_dir, n_runs=1, status=row.get("Status"),
                         score=row.get("Score"))
        self.ui_q.put(("sim_done", (row, None)))
    except Exception as e:
        import traceback
        self.ui_q.put(("sim_done", (None, f"{e}\n" + traceback.format_exc()[-1200:])))
```

```
    threading.Thread(target=work, daemon=True).start()
```

PLAN

run_plan() - one campaign engine for CLI and GUI

Why. The CLI campaign command and the GUI Campaigns tab must behave identically - same budgets, same deadlines, same results.csv - or engineers would see two different products.

How. run_plan takes the planner output, fans it out over N worker directories and gives every scenario its own time budget with a hard deadline; it accepts resume=True to continue an existing file and streams 'scenario_start' / 'scenario_done' events to a callback.

Used for: launcher.py campaign, the GUI Start campaign, self-check-only runs.

aero_auto.py - run_plan()

```
def run_plan(plan, mains, flaps_pool, check_only=False, no_check=False, resume=True, on_event=None):
    """Run every scenario described by `plan`, with per-scenario budgets, self-checks and resume.

    Shared by the command line and the GUI so both behave exactly the same. `on_event(kind, payload)`
    is an optional progress callback; kinds: "scenario_start" -> (index, total, scenario),
    "scenario_done" -> (scenario, status, runs). Returns the per-scenario (scenario, status, runs) list.
    """
    global WORKERS, REG_ENFORCE
    WORKERS = max(1, int(plan.get("workers") or WORKERS))
    REG_ENFORCE = plan.get("reg_enforce", REG_ENFORCE)
    apply_plan_numbers(plan)
    init_grid()
    scs = plan_scenarios(plan)
    t_end = time.time() + plan["hours"] * 3600
    overview = []
    for i, sc in enumerate(scs):
        if INTERRUPT.is_set():
            overview.append((sc, "not started (stopped)", 0))
            if on_event:
                on_event("scenario_done", (i, sc, "not started (stopped)", 0))
            continue
        budget = (t_end - time.time()) / 3600 / (len(scs) - i)
        if budget < 0.005 and not check_only:
            overview.append((sc, "not started (no time left)", 0))
            if on_event:
                on_event("scenario_done", (i, sc, "not started (no time left)", 0))
            continue
        if on_event:
            on_event("scenario_start", (i, len(scs), sc))
        status, n = run_scenario(sc, max(budget, 0.01), mains, flaps_pool, plan, check_only, no_check, resume)
        overview.append((sc, status, n))
        if on_event:
            on_event("scenario_done", (i, sc, status, n))
    print("\n" + "=" * 78 + "\nALL DONE")
    for sc, status, n in overview:
        print(f"  {scenario_label(sc, plan.get('chord')):<60} {status}" + (f", {n} runs" if n else "") + f"\n      {s
    return overview
```

RESUME/CSV

read_results_csv() - legacy-format recovery

Why. Real result files on this machine predate the 55-column format; a resume must not drop 20,000+ historical rows or re-number the file.

How. read_results_csv detects old 31/47-column files, upgrades them to the current column set with sensible defaults for missing columns, and normalize_results_file backs up the original before writing - so the resumed campaign continues the existing RUN_COUNTER and appends, it never overwrites.

Used for: the Results tab, launcher report/targets, campaign resume.

aero_auto.py - read_results_csv()

```
def read_results_csv(path):
    """Reads a results.csv of any vintage without dropping rows.

    Older files have a narrower header than the current FIELDS, and an upgrade in the middle of a
    file can leave rows of two different widths (the old header stays, newer rows carry the current
```

```
    FIELDS). A plain read_csv(on_bad_lines="skip") silently discards the wider rows, so this splits
    the file into groups by field count and parses each group with the column names that belong to
    it (the file header for header-width rows, FIELDS for current-width rows). The groups are then
    concatenated, so every row is kept."""
    if not path or not os.path.exists(path):
        return pd.DataFrame()
    try:
        with open(path, "r", newline="", errors="ignore") as f:
            rows = [r for r in csv.reader(f) if r and any(str(x).strip() for x in r)]
    except OSError:
        return pd.DataFrame()
    if not rows:
        return pd.DataFrame()
    header, data = rows[0], rows[1:]
    groups = {}
    for r in data:                                      # preserve first-seen order of widths
        groups.setdefault(len(r), []).append(r)
    frames = []
    odd = []
    for width, rws in groups.items():
        buf = io.StringIO()
        w = csv.writer(buf)
        if width == len(header):
            w.writerow(header)
            w.writerows(rws)
            buf.seek(0)
            frames.append(pd.read_csv(buf, on_bad_lines="skip", low_memory=False))
        else:
            if width == len(FIELDS):
                cols = FIELDS
            elif width == len(FIELDS) - 1:
                cols = FIELDS[:width]                   # previous layout (before Speed_ms was appended)
            elif width <= len(header):
                cols = header[:width]                   # trust the file's own header names
            else:
                cols = FIELDS[:width]
                odd.append(width)
            w.writerows(rws)
            buf.seek(0)
            frames.append(pd.read_csv(buf, header=None, names=cols, on_bad_lines="skip", low_memory=False))
    if odd:
        print(f"(warning: {path} has rows of unexpected width {sorted(set(odd))}; "
              f"read with a best-effort column mapping - please review the file)")
    if not frames:
        return pd.DataFrame(columns=header)
    d = pd.concat(frames, ignore_index=True, sort=False)
    return d.loc[:, ~d.columns.duplicated()]
```

CAMP-UI

on_campaign() - campaigns through the queue

Why. A GUI campaign must stream live per-scenario progress into the monitor panes without locking the window and without reading Tk from the worker.

How. on_campaign snapshots every input on the main thread, then runs run_plan in a worker that only pushes 'cam_evt' / 'cam_done' messages to the queue; the _drain_ui loop updates the scenarios tree, live-best tree, stats and log. A thread-safe stop flag and 'skip scenario' set per-scenario cancellation.

Used for: the Campaigns tab Start / Stop / Skip / Self-check buttons.

aero_ui.py - App.on_campaign()

```
def on_campaign(self, check_only=False):
    if self.cam_active:
        self.camp("a campaign is already running"); return
    got = self._collect_campaign_plan(log_fail=not check_only)
    if got is None:
        return
    plan, mains, flaps = got
    if check_only:
        plan = dict(plan, hours=0.0)                # self-check only, no real budget
    self.cam_plan = plan
    self.cam_total = plan["hours"] * 3600
```

```
    self.cam_workers_n = max(1, int(plan["workers"]))
    self.cam_t0 = time.time()
    self.cam_runs_base = len(A.RESULTS)
    self.cam_scen_idx = 0
    self.cam_active = True
    self.cam_start_btn.configure(state="disabled")
    self.cam_stop_btn.configure(state="normal")
    self.cam_skip_btn.configure(state="normal")
    self.cam_prog["value"] = 0
    self.cam_scen_tree.delete(*self.cam_scen_tree.get_children())
    self.cam_stats.set("self-checking…" if check_only else "starting…")
    self.cam_check_only = check_only
    if check_only:
        out_dir = os.path.expanduser(self.cam_out.get())
        self._camp_run_dir = None
    else:
        label = f"{plan['role']}_{plan['flow']}"
        try:
            self._camp_run_dir = A.new_run_dir("campaign", label, base=self._lib_base())
            A.write_run_meta(self._camp_run_dir, kind="campaign", label=label,
                             hours=plan["hours"], workers=plan["workers"],
                             role=plan["role"], flow=plan["flow"],
                             heights=plan["heights"], mains=mains, flaps=flaps,
                             n_scenarios=len(A.plan_scenarios(plan)), n_runs=0)
        except Exception as e:
            self._camp_run_dir = None
            self.camp(f"could not create run folder: {e}")
        out_dir = self._camp_run_dir or os.path.expanduser(self.cam_out.get())
    run_check = self.cam_check.get()
    resume = self.cam_resume.get()
    A.STOP.clear()
    A.INTERRUPT.clear()

    def on_event(kind, payload):
        self.ui_q.put(("cam_evt", (kind, payload)))

    def work():
        import traceback
        A.OUT_DIR = out_dir
        buf = _QueueWriter(self.ui_q, "camp")
        old = sys.stdout
        sys.stdout = buf
        try:
            A.run_plan(plan, mains, flaps,
                       check_only=check_only, no_check=not run_check,
                       resume=resume, on_event=on_event)
            self.ui_q.put(("cam_done", True))
        except Exception:
            traceback.print_exc()
            self.ui_q.put(("cam_done", False))
        finally:
            sys.stdout = old
    threading.Thread(target=work, daemon=True).start()
```

VIEW

load_results_cache() / sortable-filterable table

Why. With 23,000+ rows a flat tree is useless; users need to search and rank without leaving the tab.

How. load_results_cache reads through read_results_csv, binds sortable headers, and _apply_results_view applies the live filter plus numeric-aware sorting onto the frame before rendering up to 1000 rows; a stats line reports the current match count and summed sim time.

Used for: the Results tab tree, filter box, stats footer.

aero_ui.py - App.load_results_cache()

```
def load_results_cache(self):
    p = os.path.expanduser(self.res_file.get())
    if not os.path.exists(p):
        cand = self.discover_results()
        if not cand:
            return
```

```
        p = cand[0]
        self.res_file.set(p)
    try:
        self.results = A.read_results_csv(p)
    except Exception as e:
        self.set_detail(f"load failed: {e}"); return
    cols = [c for c in ("Run_ID", "Date", "Elements", "Main", "Cl_a0", "Cd_a0",
                        "Max_LD", "Downforce_N", "Duration_s", "Tier", "Score", "Status")
            if c in self.results.columns]
    self.tree["columns"] = cols
    self._results_cols = cols
    self._sort_col = None
    self._sort_asc = True
    for c in cols:
        self.tree.heading(c, text=c, command=lambda c=c: self._sort_results_by(c))
        self.tree.column(c, width=90 if c not in ("Main", "Run_ID", "Date") else 150, anchor="w")
    self._tree_frame = self.results.reset_index(drop=True)
    self._apply_results_view()
    n = len(self.results)
    ns = self._tree_frame
    what = os.path.dirname(p) or "."
    if "Duration_s" in ns.columns and "Date" in ns.columns:
        dur = pd.to_numeric(ns["Duration_s"], errors="coerce")
        tot = dur.sum()
        span = f"  |  sim time {tot:.0f}s across {n} runs" if tot else f"  |  {n} runs"
        dates = ns["Date"].dropna().astype(str)
        if len(dates):
            span += f"  |  first {dates.min()}  ->  last {dates.max()}"
    else:
        span = f"  |  {n} runs"
    self.set_detail(f"{n} simulations loaded from {what}{span}")
    self._refresh_csv_view()
```

SESSION

_build_menu / _save_state / _load_state

Why. Engineers live in this app; it should remember the window, the tab and every pool/heights/path selection, and no crash should take a day of context.

How. A menubar (File / View / Help with accelerators) covers the common actions; _load_state restores geometry + tab + configuration at startup, and _on_close writes them to ~/.aero_auto/ui_state.json before quitting.

Used for: window persistence; keyboard shortcuts (Ctrl+R, F5, Ctrl+Q).

aero_ui.py - App._save_state()

```
def _save_state(self):
    try:
        os.makedirs(os.path.dirname(self._state_path()), exist_ok=True)
        with open(self._state_path(), "w") as f:
            import json
            json.dump(self._capture_state(), f, indent=1)
    except Exception:
        pass
```

OPT-UI

_opt_start / _calculate_objective_score

Why. The optimizer runs iterations of real JavaFoil; it must report progress and stay stoppable, and its scorer must be callable from the worker thread.

How. _opt_start snapshots all optimizer inputs on the main thread, then the worker runs simulated candidates and pushes 'opt_prog' / 'opt_status' / 'opt_done' through the queue. _calculate_objective_score is a pure staticmethod (no Tk access) that weighs lift, efficiency, stability, height, length, gap, downforce and target-efficiency terms.

Used for: the Optimizer tab; Apply Best feeds the winner back to Configure.

aero_ui.py - App._calculate_objective_score()

```
@staticmethod
def _calculate_objective_score(row, objs, targets):
```

```
    """Pure objective-scorer for the optimizer (no Tk access - called from the worker).

    Maximizes the weighted sum of the selected objective contributions.
    """
    def num(key):
        v = row.get(key)
        try:
            v = float(v)
        except (TypeError, ValueError):
            return 0.0
        return v if math.isfinite(v) else 0.0

    terms = []
    if "lift" in objs:                      # more negative Cl_a0 = more downforce
        terms.append(num("Cl_a0") * -1.0)
    if "efficiency" in objs:                # Max_LD (positive)
        terms.append(0.2 * num("Max_LD"))
    if "stability" in objs:                 # minimize |Cl variation around alpha 0|
        var = (num("Cl_am2") - num("Cl_a0")) ** 2 + (num("Cl_ap2") - num("Cl_a0")) ** 2
        terms.append(-math.sqrt(var))
    if "height" in objs:                    # minimize stack height (mm)
        terms.append(-num("Stack_Height_mm") / 100.0)
    if "length" in objs:                    # minimize stack length (mm)
        terms.append(-num("Stack_Length_mm") / 100.0)
    if "gap" in objs:                       # match the target main->F1 gap
        g = num("Main_to_F1_Gap_pct")
        if g <= 0 and len(objs) == 1:
            return 0.0
        terms.append(-abs(g - targets["target_gap"]))
    if "downforce" in objs:                 # maximize downforce (N)
        terms.append(num("Downforce_N") / 300.0)
    if "target_eff" in objs:                # Max_LD penalised by distance to target Cl
        delta = abs(num("Cl_a0") + targets["target_cl"])
        terms.append(0.2 * num("Max_LD") / (1.0 + delta))
    return float(sum(terms))
```

STORE

install_airfoil / list_store

Why. Users asked to manage profiles inside the app without hunting .dat files on disk.

How. The GUI imports a .dat into APP_DATA_DIR/airfoils under a type-in name (sanitised with a regex), reloads the combos, and can list/remove store entries; the store folder auto-created on first use.

Used for: the Airfoils tab and every airfoil combo in the app.

aero_auto.py - install_airfoil()

```
def install_airfoil(src, name=None, store=None):
    """Copy a .dat profile into the internal airfoil store under a chosen name.

    Returns (store_name, path_to_dat). The profile then appears in every airfoil
    combo of the GUI without you having to manage where files live."""
    store = store or AIRFOIL_STORE
    os.makedirs(store, exist_ok=True)
    base = os.path.splitext(os.path.basename(src or ""))[0]
    name = re.sub(r"[^A-Za-z0-9 _\\-]", "", (name or base).strip()).strip() or "airfoil"
    dst = os.path.join(store, name + ".dat")
    shutil.copy(src, dst)
    return name, dst
```

GRAPHS

on_gen_graphs / refresh_graph_files

Why. All generated data - polars, geometry, velocity fields - should be reachable from one tab.

How. on_gen_graphs renders polar/geometry/field PNGs for the selected results row and hands the resulting path to refresh_graph_files, which also discovers <result-dir>/graphs/*.png and shows a Pillow thumbnail preview inline.

Used for: the Graphs tab; the Results tab's 'Graphs for selected run' button.

aero_ui.py - App.on_gen_graphs()

```
def on_gen_graphs(self):
    if R is None:
        self.log("aero_report not importable"); return
    r = self._selected_row()
    if r is None:
        self.set_detail("select a row in the table first, then hit Graphs for selected run")
        return
    self._gen_graphs(r, switch=True)
```

REPORT/VIEW

on_report / on_targets

Why. The user should not leave the GUI to read the numbers they just generated.

How. Both launch the corresponding headless tool as a subprocess-style call inside a worker thread with stdout redirected into the queue; on_report points at aero_report.main.

Used for: the Report and Show-targets buttons.

aero_ui.py - App.on_report()

```
def on_report(self, path=None):
    if R is None:
        self.camp("aero_report not importable"); return
    path = path or os.path.join(os.path.expanduser(self.cam_out.get()), "results.csv")
    if not os.path.exists(path):
        self.camp(f"no results at {path}"); return
    self._run_report(path)
```

DRAIN

_drain_ui

Why. Decouples widget mutation from the worker threads completely.

How. A 50 ms after() loop pops the queue and applies 'log', 'camp', 'load', 'aflog', 'refreshgr', 'cam_evt', 'cam_done' and the 'opt_*' messages to the right widgets on the Tk thread; re-arms itself each pass. 'refreshgr' can carry a results-file path so the Graphs tab switches to the file that was just written.

Used for: thread-safety of the entire UI.

aero_ui.py - App._drain_ui()

```
def _drain_ui(self):
    # Never let one bad message (or a UI callback that raises) kill the update loop: the timer is
    # always rescheduled in the finally block, so the window cannot freeze permanently.
    try:
        while True:
            try:
                kind, payload = self.ui_q.get_nowait()
            except queue.Empty:
                break
            try:
                if kind == "log":
                    self._append_log(self.status, payload)
                elif kind == "camp":
                    self._append_log(self.cam_log, payload,
                                     follow=self.cam_follow.get())
                elif kind == "load":
                    self.load_results_cache()
                    self.refresh_graph_files()
                elif kind == "cam_evt":
                    self._cam_evt(*payload)
                elif kind == "cam_done":
                    self._cam_done(payload)
                elif kind == "aflog":
                    self._append_log(self.af_log, payload)
                elif kind == "refreshgr":
                    self.refresh_graph_files(payload or None)
                elif kind == "opt_log":
                    self._append_log(self.opt_log, payload)
                elif kind == "opt_prog":
                    self.opt_progress["value"] = payload
                elif kind == "opt_status":
                    self.opt_status.set(payload)
```

```
            elif kind == "opt_done":
                self._handle_opt_done(payload)
            elif kind == "sim_done":
                self._sim_done(payload)
        except Exception as e:              # a bad payload must not kill the drain loop
            try:
                self._append_log(self.status, f"UI update error ({kind}): {e}")
            except Exception:
                pass
except queue.Empty:
    pass
except Exception as e:                      # never let the drain loop die
    try:
        self._append_log(self.status, f"UI update error: {e}")
    except Exception:
        pass
try:
    self._update_campaign_progress()
    cnt = getattr(self, "_drain_cnt", 0) + 1
    self._drain_cnt = cnt
    if self.cam_active and cnt % 20 == 0:
        self._refresh_live_campaign()
except Exception:
    pass
finally:
    if self._drain_timer_running:
        self.after(50, self._drain_ui)
```

8 render_field.py + launcher.py - CLI and packaging entry points

FIELD-CLI

render_field.main()

Why. The field images needed a non-GUI path for scripting and CI.

How. Reads --main/--flap/--alpha/--ground-height, grabs the best CI_a0 it can find in existing results (fallback -2.0), runs compute_field and writes a dark-free viridis contour PNG with dimensionless axes.

Used for: headless field renders from any configuration.

render_field.py - main()

```
def main(argv=None):
    ap = argparse.ArgumentParser(prog="render_field")
    ap.add_argument("--main", required=True, help="main element airfoil name")
    ap.add_argument("--flap", action="append", default=[],
                    help="flap as 'airfoil,scale,angle,x,y' (repeatable up to 3)")
    ap.add_argument("--alpha", type=float, default=0.0, help="angle of attack, deg")
    ap.add_argument("--ground-height", type=float, default=None,
                    help="main height above ground as percent of chord (rows the wing up; ground at y=0)")
    ap.add_argument("--chord-mm", type=float, default=300.0, help="chord in mm for the metric overlay")
    ap.add_argument("--out", default="field.png")
    ap.add_argument("--nx", type=int, default=300)
    ap.add_argument("--ny", type=int, default=300)
    args = ap.parse_args(argv)

    A.AIRFOILS.update(A.load_all(A.AIRFOIL_DIR))
    flaps = []
    for f in args.flap:
        bits = f.split(",")
        if len(bits) != 5:
            raise SystemExit("flap must be 'airfoil,scale,angle,x,y'")
        name, s, a, x, y = bits
        flaps.append((name.strip(), float(s), float(a), float(x), float(y)))
    cfg = (args.main, tuple(flaps))

    outs = A.build_outlines(cfg, ground_h=args.ground_height)
    if args.ground_height:
        ground_h = args.ground_height
```

```
    else:
        ground_h = None
    # circulation estimate from the most recent JavaFoil Cl if a report cache exists
    cl = _best_cl(cfg)
    print(f"using Cl@{args.alpha:g} deg = {cl:.3f} (from results, or -2.0 fallback)", file=sys.stderr)

    X, Y, U, V = FF.compute_field(outs, alpha=args.alpha, ground_h=ground_h, Cls=[cl] * len(outs),
                                  nx=args.nx, ny=args.ny)
    sp = np.hypot(U, V)
    sp = np.clip(sp, 0, 3)

    fig, ax = plt.subplots(figsize=(8.5, 6), dpi=110)
    cm = ax.pcolormesh(X, Y, sp, cmap="viridis", vmin=0, vmax=3, shading="auto")
    fig.colorbar(cm, ax=ax, label="|V| / U (clipped to 3; white = inside the wing)")
    for o in outs:
        ax.plot(np.append(o[:, 0], o[0, 0]), np.append(o[:, 1], o[0, 1]), color="black", lw=1.4)
    for x0, y0 in _seed_lines(X, Y):
        pts = FF.streamlines(X, Y, U, V, seeds=[(x0, y0)], n=220, ds=0.02)
        if pts and len(pts[0]) > 2:
            line = np.asarray(pts[0], float)
            ax.plot(line[:, 0], line[:, 1], color="white", lw=0.6, alpha=0.55)
    if args.ground_height:
        ax.axhline(0, color="red", lw=1.4)
    ax.set_aspect("equal")
    ax.set_xlabel("x (chords)")
    ax.set_ylabel("y (chords)")
    ax.set_title(f"{cfg[0]} + {len(cfg[1])} flaps  alpha {args.alpha:g} deg  "
                 f"(chord {args.chord_mm:g} mm = shown distance)")
    fig.tight_layout()
    fig.savefig(args.out, dpi=110)
    print("wrote", args.out)
    return 0
```

LAUNCHER

launcher.launcher()

Why. One executable name across tools and platforms (ui/campaign/report/targets/field).

How. Argparse sub-tool selection with parse_known_args forwarding, plus environment overrides AERO_JAVA_DIR / AERO_AIRFOILS / AERO_OUT before dispatch; 'aero-ui' and 'aero' console scripts from pyproject.toml call it.

Used for: run_ui.sh / run_ui.bat / desktop entry and the installed CLI.

launcher.py - launcher()

```
def launcher():
    p = argparse.ArgumentParser(prog="aero", description="aero_auto toolchain entry point")
    p.add_argument("tool", nargs="?", default="ui", choices=["ui", "campaign", "report", "targets", "field"])
    args, rest = p.parse_known_args()
    _apply_env()
    if args.tool == "ui":
        import aero_ui
        aero_ui.main()
    elif args.tool == "campaign":
        sys.argv = ["aero_auto.py"] + rest
        A.main()
    elif args.tool == "report":
        import aero_report
        sys.argv = ["aero_report.py"] + rest
        aero_report.main()
    elif args.tool == "targets":
        import aero_targets
        sys.argv = ["aero_targets.py"] + rest
        aero_targets.main()
    elif args.tool == "field":
        import render_field
        render_field.main(rest)
    return 0
```

9 Validation and performance

Reported runtime from an x86-64 laptop, Python 3.14, single-threaded:

Operation	Cost
single-element JavaFoil polar (25 alphas)	~ 5 s
four-element stack polar	~ 20-60 s
300x300 four-element ground field	~ 19-33 s
3-config FW ground report incl. all graphs	~ 71-92 s total
3-config report with images off	~ 10 s

Artifact read-back checks performed while building the report layer:

- every column in results_human.csv is explained by columns_legend.csv (Status and Rank included)
- no duplicate/'.1' headers after rename_columns (mm columns carry their unit)
- overview.xlsx opens with the expected Top configs / All rows / Legend sheets
- resim_polar_curve output matched the stored Cl_a0 exactly (-1.338 on s1223)

10 Packaging and build

The same five modules ship in four forms: source kits (Linux tar.gz, Windows zip), the installed ~/Applications/aero_auto desktop app, and a standalone PyInstaller build made under wine on this machine.

Artifact	Built by	Notes
aero_auto_linux_1.1.tar.gz	build_dist.py	./run_ui.sh; python3.9+ + deps
aero_auto_windows_1.1.zip	build_dist.py	run_ui.bat; needs installed Python
aero_auto_windows_app_1.1.zip	hand-rolled zip of dist/aero_ui	self-contained exe, no Python needed
desktop app	~/.local/share/applications/aero_auto.desktop	launches run_ui.sh; icon from assets
aero_ui.exe	create_standalone.bat under wine (win python 3.12 + PyInstaller)	--onedir --windowed; assets+docs bundled via --add-data/--add-binary

Icon pipeline (make_icon.py)

Priorities: assets/Logoertblanc.png, then assets/source.png, then a generated placeholder. Pillow produces assets/icon.png (512) and a multi-size icon.ico for Windows; the GUI prefers the .ico and falls back to .png or AERO_ICON.

Windows note

Faketime is unavailable on Windows; simulate() auto-detects it and runs a bare java when missing (2.5). The .exe was built with the same flag surface documented in create_standalone.bat.

11 Changelog - everything added, and why

1.1 - added since 1.0

Addition	Why / for what
shared run_plan engine (CLI + GUI)	one campaign behaviour for launcher and the Campaigns tab; per-scenario budgets and a strict deadline; resume preserves history
legacy results.csv auto-upgrade with backup	old 31/47-column files keep their 20,000+ rows on resume instead of being dropped; the previous file is backed up before migration

Addition	Why / for what
Campaigns tab - setup + live monitor	scenarios tree with live status/runs, live best config, stats (elapsed/finished/runs-per-s), Skip scenario, Stop all, Self-check only, Copy plan to clipboard
Optimizer tab	objective-weighted hill climbing over flap geometry with a pure worker-thread scorer, live progress and Apply Best
Results tab filter/sort/stats	23,000+ rows need a live filter, numeric sorting and a match-count footer, not a wall of text
menu bar + status bar + settings persistence	keyboard access (Ctrl+R, F5, Ctrl+Q), at-a-glance environment state, and ~/.aero_auto/ui_state.json so no restart loses context
optimizer UI bridge repair	the old opt_* drain method was never scheduled in the queue loop - opt progress/status/done now ride the main _drain_ui path

1.0 - the founding feature set

Addition	Why / for what
regulatory engine (FSUK/FSG) + GUI verdict	instant legality feedback and a cheap campaign gate before JavaFoil
geometry_columns, all-mm fields	comparable audit numbers for stack/gap/clearance
force_columns (Downforce_N, Drag_N)	physical downforce/drag from dimensionless polars
ground-effect campaigns + per-height files	graph realism at fixed ride height
target report (CL*A / CD*A roofs)	the design point, not just the fastest stack
calibrated flowfield model	fast, stable images after the linear solver failed (2.1)
report pipeline + xlsx + legend + duplicate-safe names	self-explanatory deliverables from any historical CSV
--resim denser JavaFoil polar	smooth top-config curves; absolute jar path
threaded Tk GUI, queue bridge	non-blocking UI without Tk-thread corruption
faketime fallback	Windows portability of the simulation runner
launcher/render_field + platform launchers	one command surface on Linux and Windows
packaging: desktop entry, wine exe, icon pipeline	run it as an app, ship it as an exe